UNIVERSIDAD DE LOS ANDES
DEPARTAMENTO DE INGENIERIA DE SISTEMAS Y COMPUTACIÓN

LABORATORIO: ANÁLISIS CAPA DE TRANSPORTE Y SOCKETS

ISIS 3204 – REDES Y SERVICIOS DE COMUNICACIONES

Yuri Andrea Pinto Rojas
Nathalia Alexandra Quiroga Alfaro

Grupo 8
Juan Pablo Delgado Martínez - 202212285
Carlos David Rodríguez Barrios - 202310137
Daniel Eduardo Rivera Acevedo - 202123493

2026-10

TABLA DE CONTENIDO

Introducción 3
Procedimiento 3
2.1 Implementación con sockets TCP 3
2.1.1 Arquitectura general 3
2. 1. 2 implementación en Assembler 6
2.2 Implementación con sockets UDP 10
2.2.1 Arquitectura general 10
2.2.2 Implementación en Assembler 12
2.3 Simulación de perdida de paquetes. 14
3. Pruebas y Configuración del Entorno 15
4. Resultados de Ejecución 16
4.1 Evidencia de la ejecución en TCP 16
4.2 Evidencia de la ejecución en UDP 16
4.3 Análisis de Tráfico en Wireshark 17
4.4 Simulación de Pérdida de Paquetes (UDP) 18
4.5 Comparación entre TCP y UDP 19
5. Análisis y Respuestas 19
6. Link GitHub 21

Introducción

En el laboratorio análisis de la capa de transporte y sockets se desarrolló un sistema de comunicación basado en el modelo publicación-suscripción. El objetivo fue comparar el comportamiento de TCP y UDP en aspectos como confiabilidad, orden de entrega y perdida de mensajes, además de observar su transporte mediante pruebas y capturas de tráfico.

El escenario planteado corresponde a un sistema de noticias deportivas en tiempo real, en el que los publicadores envían eventos de un partido, el broker recibe y distribuye la información y los suscriptores reciben las actualizaciones. A partir de este movimiento de información, se implementaron dos versiones del sistema. Una usando TCP y la otra usando UDP, con el fin de comparar de forma práctica las características de ambos protocolos.

Procedimiento

2.1 Implementación con sockets TCP

En esta sección se describe la implementación del sistema de publicación–suscripción utilizando **sockets TCP en lenguaje C**, conformado por tres componentes principales: **broker, publisher y subscriber**. Este sistema simula el comportamiento del protocolo TCP, aprovechando sus características de confiabilidad, entrega ordenada y control de conexión.

2.1.1 Arquitectura general

El sistema sigue el modelo **publicación–suscripción**, donde:

- El **broker** actúa como servidor central, recibiendo conexiones TCP de múltiples clientes.
- Los **publishers** se conectan al broker y envían mensajes asociados a un tema.
- Los **subscribers** se conectan al broker y reciben mensajes de los temas a los que están suscritos.

La comunicación se realiza mediante **sockets TCP (SOCK_STREAM)**, lo que implica que:

- Se establece una conexión previa (3-way handshake).
- Los mensajes llegan completos y en orden.
- El sistema maneja automáticamente retransmisiones y control de flujo.

<u>Uso de librerías en C</u>

La implementación utiliza principalmente librerías estándar de C y librerías del sistema POSIX para manejo de sockets. Aunque no son "externas" como frameworks, **sí son librerías no nativas del núcleo del lenguaje C**, ya que pertenecen al sistema operativo (Linux).

Librerías utilizadas y su función

- <stdio.h>: Permite la entrada y salida estándar. Se usa para imprimir mensajes en consola (printf) y reportar errores (perror).
- <stdlib.h>: Maneja utilidades generales como:
    - exit() para terminar el programa en caso de error.
    - malloc() y free() para gestión de memoria dinámica (usado en el broker).
- <string.h>: Manejo de cadenas:
    - strlen() para longitud de mensajes.
    - strcmp() para comparar comandos (PUB/SUB).
    - strncpy() y memset() para manipulación de buffers.
- <unistd.h>: Proporciona acceso a llamadas del sistema:
    - close() para cerrar sockets.
    - read() y write() (alternativas a recv/send).
    - sleep() para simular intervalos entre mensajes.
- <sys/socket.h>
- **Librería clave para la simulación de TCP**, contiene la API de sockets:
    - socket() → crea el socket.
    - bind() → asocia dirección y puerto.
    - listen() → habilita recepción de conexiones.
    - accept() → acepta conexiones entrantes.
    - connect() → inicia conexión cliente-servidor.
    - send() / recv() → envían y reciben datos.
- <netinet/in.h>: Define estructuras para direcciones IP:
    - struct sockaddr_in
    - htons() para convertir puertos a formato de red.
- <arpa/inet.h>: Conversión de direcciones IP:
    - inet_pton() convierte IP texto → binario.
    - inet_ntoa() convierte IP binario → texto.
- <pthread.h> *(usada en el broker)*
  Permite manejo de concurrencia:
    - pthread_create() crea hilos.
    - pthread_mutex_* evita condiciones de carrera.

Implementación del Broker

El broker es el componente central y funciona como un **servidor TCP concurrente**.

Flujo principal

1. **Creación del socket:** socket(AF_INET, SOCK_STREAM, 0), define un socket TCP.
2. **Configuración del socket:** setsockopt(... SO_REUSEADDR ...), permite reutilizar el puerto sin esperar.
3. **Asociación a puerto:** bind(), se asocia al puerto 8080.
4. **Escucha de conexiones:** listen(), habilita la cola de conexiones entrantes.

5. **Aceptación de clientes:** accept(), el kernel completa automáticamente el 3-way handshake TCP.
6. **Manejo concurrente:** Cada cliente se atiende en un hilo independiente con pthread_create()

Lógica de funcionamiento

- El broker identifica clientes según el primer mensaje:
  - "PUB <tema>" → publicador
  - "SUB <tema>" → suscriptor
- Mantiene una **tabla de suscriptores** protegida con mutex.
- Cuando recibe un mensaje de un publisher: recv(), lo reenvía a todos los suscriptores del mismo tema usando send(), esto implementa la lógica de distribución del modelo pub-sub.

<u>Implementación del Publisher</u>

El publisher actúa como cliente TCP que envía eventos.

Flujo Principal

1. Crear socket: socket()
2. Conectarse al broker: connect(), aquí ocurre el handshake TCP.
3. Identificarse: send("PUB <tema>\n")
4. Enviar mensajes: send()
5. Cerrar conexión: close(), esto genera el cierre TCP (FIN).

Comportamiento

- Envía 12 mensajes simulando eventos deportivos.
- Introduce pausas (sleep) para simular tiempo real.

<u>Implementación del Subscriber</u>

El subscriber recibe mensajes en tiempo real.

Flujo Principal

1. Crear socket: socket()
2. Conectarse al broker: connect()
3. Suscribirse: send("SUB <tema>\n")
4. Recibir mensajes: recv()
5. Cierre: close()

Recepción de datos:

- Lee byte a byte hasta encontrar \n.

- Esto permite reconstruir mensajes completos.
- TCP garantiza que los datos llegan **sin pérdida y en orden**.

Simulación del comportamiento de TCP

Aunque no se implementa TCP desde cero, el programa **simula su comportamiento usando la API de sockets**, evidenciando:

- **Handshake (3-way handshake):** Implícito en connect() y accept().
- **Entrega confiable:** TCP asegura que los mensajes enviados con send() llegan correctamente.
- **Orden de mensajes:** Garantizado por el protocolo.
- **Control de flujo:** Manejado por el kernel (no visible en código).
- **Cierre de conexión:** Realizado con close() (proceso FIN).

2. 1. 2 implementación en Assembler

En esta sección se describe la implementación del sistema de publicación–suscripción utilizando **lenguaje ensamblador (Assembly x86_64, sintaxis AT&T)** sobre Linux. Esta versión es equivalente a la implementación en C, pero permite observar con mayor detalle cómo interactúa el programa con el sistema operativo a bajo nivel.

El sistema está compuesto por tres programas:

- broker_tcp.s
- publisher_tcp.s
- subscriber_tcp.s

Estos utilizan llamadas a funciones de librerías del sistema (libc) para implementar comunicación mediante **sockets TCP**.

Arquitectura y simulación de TCP

El protocolo TCP se simula usando llamadas al sistema a través de libc, lo que permite evidenciar:

- Establecimiento de conexión → connect() / accept()
- Envío confiable → send()
- Recepción ordenada → read() / recv()
- Cierre de conexión → close()

El control de errores, retransmisión, orden y control de flujo son manejados por el kernel de Linux.

Convención de llamadas (clave en Assembly)

Se usa la convención System V AMD64, donde:

- Argumentos:
  - rdi, rsi, rdx, rcx, r8, r9
- Valor de retorno:
  - rax

Ejemplo:

```
movl $2, %edi     # argumento 1
movl $1, %esi     # argumento 2
call socket       # llamada a función
```

Librerías utilizadas y su función

Aunque no se incluyen explícitamente como en C, el programa usa funciones de **libc**:

Funciones de red (sockets)

- socket(): Crea un socket TCP.
- connect(): Inicia la conexión con el servidor (3-way handshake).
- bind() *(solo broker):* Asocia el socket a un puerto.
- listen() *(solo broker):* Pone el socket en modo escucha.
- accept() *(solo broker):* Acepta conexiones entrantes.
- send(): Envía datos por el socket.
- read(): Recibe datos del socket.
- close(): Cierra la conexión TCP.

<u>Implementación del Publisher (publisher_tcp.s)</u>

El publisher actúa como cliente TCP que envía mensajes al broker.

Flujo Principal

1. Creación del socket

```
movl $2, %edi      # AF_INET
movl $1, %esi      # SOCK_STREAM (TCP)
call socket
```

2. Configuración de dirección: Se construye manualmente la estructura sockaddr_in:

```
movw $2, (%rbx)        # AF_INET
movw $0x8813, 2(%rbx)  # puerto 5000
```

3. Conversión de IP : call inet_pton

4. Conexión al broker : call connect, aquí ocurre el **3-way handshake TCP** automáticamente.

5. Envío de mensajes: call send

6. Lectura desde teclado: call fgets
7. Cierre de conexión: call close

Comportamiento

- Lee mensajes desde stdin.
- Calcula su longitud con strlen.
- Los envía al broker mediante send.
- Simula transmisión en tiempo real.

Implementación del Subscriber (subscriber_tcp.s)

El subscriber recibe mensajes enviados por el broker.

Flujo Principal

1. Crear socket → socket()
2. Conectarse → connect()
3. Recibir datos: call read
4. Mostrar mensajes: : call printf
5. Cerrar conexión → close()

Recepción de datos

- read() llena un buffer con datos del socket.
- Se agrega manualmente el carácter \0: movb $0, (%rax,%rcx,1). Esto convierte el buffer en un string válido en C.

Implementación del Broker (broker_tcp.s)

El broker es el componente más complejo, ya que maneja múltiples clientes simultáneamente.

Flujo principal

1. **Crear socket** : call socket
2. **Configurar dirección:**
   movw $2, (%rbx) # AF_INET
   movw $0x8813, 2(%rbx) # puerto
3. **Bind:** call bind
4. **Listen:** call listen
5. **Aceptar conexiones:** call accept

Multiplexación con select()

El broker utiliza: call select

Esto permite:

- Manejar múltiples clientes sin hilos
- Detectar sockets con actividad

Estructura fd_set (manual)

Como no hay macros en Assembly, se implementan manualmente:

- FD_SET: set[fd/64] |= (1 << (fd % 64))
- FD_ISSET: set[fd/64] & (1 << (fd % 64))

Esto permite monitorear múltiples sockets.

Recepción de mensajes

call read si:

- read == 0 → cliente desconectado
- read > 0 → mensaje válido

Broadcast

El broker reenvía mensajes a todos los clientes:call send. Excluyendo el cliente que envió el mensaje.

Simulación del comportamiento TCP

A través de estas llamadas, se evidencia:

1. Handshake TCP

- connect() y accept() lo ejecutan automáticamente.

2. Confiabilidad

- send() garantiza entrega correcta.
- Si falla, el kernel retransmite.
-

3. Orden

- Los mensajes se reciben en el orden enviado.

4. Control de flujo

- Gestionado por el kernel (no visible en Assembly).

5. Cierre de conexión

- close() inicia el cierre TCP (FIN).

2.2 Implementación con sockets UDP

En esta sección se explica la implementación del sistema de publicación-suscripción usando sockets UDP. El código está compuesto por tres programas: broker, publisher y subscriber. A diferencia de TCP, en UDP no existe conexión permanente entre las partes, sino que la comunicación se realiza mediante datagramas independientes. Por esta razón, UDP tiene menor overhead, pero no garantiza la entrega, ni el orden ni la recuperación automática de mensajes enviados.

2.2.1 Arquitectura general

Como se mencionó anteriormente, el sistema sigue la siguiente estructura:

- El **broker** recibe los mensajes y los distribuye
- El **publisher** envía mensajes al broker
- El **subscriber** se registra para recibir mensajes reenviados por el broker.

La comunicación se realiza mediante sockets **UDP (SOCK_DGRAM)**, por lo que:

- No hay establecimiento de conexión previo.
- Cada mensaje se envía como un datagrama.
- No hay garantia de entrega
- No hay garantía de orden
- Si se necesita confirmación, debe implementarse a nivel de aplicación.

Uso de Librearías en C

Se usan librerías estándar y librerías del sistema para sockets:

- **<stdio.h>**: Permite imprimir mensajes y errores.
- **<stdlib.h>:** Se usa para constantes de salida del programa.
- **<string.h>:** Se usa para inicializar estructuras y copiar cadenas.
- **<unistd.h>:** Permite cerrar sockets.
- **<sys/socket.h>:** Contiene funciones como socket(), bind(), sendto() y recvfrom().
- **<netinet/in.h> y <arpa/inet.h>:** Permiten manejar direcciones IP y puertos.
- **<sys/select.h>:** Se usa en el publisher para esperar el ACK.

Estructura mensajes

Para la comunicación UDP se definió una estructura común llamada Packet, que contiene:

- **type:** Indica el tipo de mensaje;
- **seq:** Guarda el número de secuencia;
- **payload:** Contiene el mensaje.

También se definieron distintos tipos de mensaje para identificar publicaciones, suscripciones, datos enviados a suscriptores y confirmaciones.

Implementación del broker

Este es el componente central de todo el sistema (para UDP), su funcion es recibir mensajes, registrar suscriptores y reenviar publicaciones.

Flujo Principal

1. Se crea el socket con "socket(AF_INET, SOCKET_DRAM, 0)"
2. Se configura la dirección del servidor.
3. Se asocia el socket al puerto con "bind()".
4. El broker entra en un ciclo de recepcion con "recvfrom()".
5. Según el tipo de mensaje recibido, se realiza la siguiente acción:
   - Se registra un suscriptor.
   - Se procesa una publicación.
   - Se reporta un tipo de mensaje no soportado.

El borker mantiene un arreglo de suscriptores activos. Cuando recibe un mensaje, guarda la dirección del cliente si no estaba registrada. Cuando recibe una publicación, imprime el mensaje recibido, envía un ACK al publisher y luego reenvía el contenido a todos los suscriptores.

Implementación del Publisher

Este componente envía los mensajes al broker

Flujo Principal

1. Crea un socket UDP con "socket()".
2. Configura la dirección del broker.
3. Lee los mensajes.
4. Para cada mensaje leído, realiza la siguiente acción:
   - Se crea un paquete tipo publicación.
   - Asigna un número de secuencia.
   - Copia el mensaje en el "payload".
   - Envía el mensaje al broker usando "sendto()"

Como se mencionó anteriormente, UDP no garantiza por sí mismo la entrega del mensaje, por lo que se implementó un mecanismo de confirmación a nivel de aplicación. Al realizar un envió, el publisher espera un ACK usando "select()" durante un tiempo limitado. Si ese ACK no llega, el mensaje se reenvía nuevamente hasta alcanzar un límite máximo de intentos.

Implementación del Subscriber:

Este componente del UDP se encarga de registrarse ante el broker y recibir los mensajes.

Flujo Principal

1. Crea un socket UDP con "socket()".
2. Hace "bind()" sobre una dirección.

3. Configura la dirección del broker.
4. Envía un mensaje de suscripción con **“sendto()”.**
5. Entra en un ciclo de recepción con **“recvfrom()”.**
6. Cuando recibe la información, muestra un numero de secuencia y el mensaje recibido.

Este componente no establece una conexión persistente con el broker. Solo envía un datagrama de suscripción y desde ese momento queda registrado para recibir publicaciones reenviadas.

2.2.2 Implementación en Assembler

Para el modelado de UDP, se realizó una nueva implementación del sistema de publicación-suscripción utilizando el lenguaje Assembly sobre Linux. Como el código es una versión equivalente a la implementación en C, se evita repetir el mismo proceso de implementación. Se resalta la diferencia de código.

Funciones utilizadas

En assembler no se usan directamente librearías como en C, sino que se invocan funciones de “libc” siguiendo la convención de llamadas de Linux.

- **socket():** crea el socket UDP
- **bind():** asocia el socket a un puerto
- **sendto():** envía datagramas
- **recvfrom():** recibe datagramas
- **select():** espera datos con timeout
- **close():** cierra el socket

Estas funciones se llaman cargando los argumentos en registros (rdi, rsi, rdx, etc) y usando call.

<u>Implementación del broker (broker_udp.s)</u>

El broker recibe los mensajes y los redistribuye.

Ejemplo 1: Inicialización y procesamiento de mensajes.

```
call socket
call bind
call recvfrom      # recibe mensajes
call sendto        # envía ACK o datos
```

En assembler, cada call invoca directamente una función de libc.

Ejemplo 2: Identificación del tipo de mensaje

```
movl paquete(%rip), %eax
cmpl $1, %eax
je .manejar_suscripcion
cmpl $2, %eax
je .manejar_publicacion
```

Aquí se accede a memoria usando direccionamiento relativo (%rip). Luego se compara el valor con constantes y se usan saltos (je) para controlar el flujo del programa.

Ejemplo 3: Construcción de mensaje DATA para suscriptores

```
movl $3, entrega(%rip)          # MSG_TYPE_DATA
movl paquete+4(%rip), %eax
movl %eax, entrega+4(%rip)
```

En assembler, la construcción de estructuras se hace manualmente, escribiendo directamente en posiciones de memoria. No existen structs como en C, solo offsets.

<u>Implementación del Publisher (publisher_udp.s)</u>

El publisher envía mensajes al broker y espera confirmación.

Ejemplo 1: Creacion del socket y envió y recepción

```
movl $2, %edi      # AF_INET
movl $2, %esi      # SOCK_DGRAM
call socket

call sendto        # envío del mensaje
call recvfrom      # espera de ACK
```

Los parámetros se pasan en registros antes del call.

Ejemplo 2: Espera con timeout (select)

```
movl sockfd(%rip), %edi
incl %edi
leaq readfds(%rip), %rsi
xorl %edx, %edx
xorl %ecx, %ecx
leaq tiempo_espera(%rip), %r8
call select
```

Aquí se observa el uso de direcciones con leaq para pasar punteros. En assembler, el programador debe manejar manualmente las estructuras usadas por funciones como select.

Ejemplo 3: Validación del ACK recibido

```
cmpl $4, paquete_ack(%rip)   # MSG_TYPE_ACK
jne .ack_no
movl paquete_ack+4(%rip), %eax
cmpl %ebx, %eax              # comparación de seq
jne .ack_no
```

La validación se realiza mediante comparaciones y saltos condicionales. No hay estructuras de control como if, todo se implementa con instrucciones de salto.

Implementación del Subscriber (subscriber_udp.s)

Se encarga de registrar y recibir los mensajes del broker.

Ejemplo 1: Creacion del socket y envió y recepción

```
call socket
call bind
call sendto        # suscripción
call recvfrom      # recepción de mensajes
```

Cada operación debe llamarse manualmente.

Ejemplo 2: Construcción del mensaje de suscripción

```
movl $1, suscripcion(%rip)      # MSG_TYPE_SUBSCRIBE
movb $'S', suscripcion+8(%rip)
movb $0, suscripcion+9(%rip)
```

Los datos se escriben directamente en memoria. Cada campo del mensaje se asigna usando offsets específicos.

Ejemplo 3: Impresión del mensaje recibido

```
movl $3, entrega(%rip)          # MSG_TYPE_DATA
movl paquete+4(%rip), %eax
movl %eax, entrega+4(%rip)
```

Antes de llamar a printf, los argumentos se cargan en los registros correspondientes.

2.3 Simulación de perdida de paquetes.

Para evidenciar el comportamiento de UDP ante fallos, se implementó una versión modificada del broker (bróker_udp_prueba.s) que simula la pérdida de mensajes de forma controlada.

Esta versión permite analizar cómo el sistema reacciona cuando no se garantiza la entrega, característica propia del protocolo UDP.

Proceso

El broker fue modificado para perder intencionalmente el primer ACK enviado al publisher y perder el primer mensaje enviado a un subscriber

Esto se controla mediante variables internas:

- movl $1, perder_primer_ack(%rip)
- movl $1, perder_primer_envio(%rip)

Estas variables actúan como banderas que permiten omitir el envío únicamente en la primera ocurrencia.

Ejemplo 1: Pérdida del ACK

```
cmpl $0, perder_primer_ack(%rip)
je .ack_normal
movl $0, perder_primer_ack(%rip)
```

En este fragmento se verifica si se debe omitir el envío del ACK. Si la bandera está activa, el ACK no se envía y luego se desactiva para continuar el flujo normal.

Ejemplo 2: Pérdida del envío al subscriber

```
cmpl $0, perder_primer_envio(%rip)
je .envio_normal
movl $0, perder_primer_envio(%rip)
```

En este fragmento se aplica la misma lógica, pero sobre el envío del mensaje a los suscriptores.

Resumen

En esta prueba, el publisher no recibe el ACK en el primer intento, así que debe volver a enviar el mensaje, y el subscriber tampoco recibe el primer mensaje que manda el broker. Después, los siguientes envíos funcionan. Esto muestra que en UDP los mensajes pueden perderse y para evitar esto se requiere más confiabilidad. En este caso, el uso de ACK y reintentos en el publisher ayuda a manejar mejor esas pérdidas y permite probar de forma controlada cómo se comporta el sistema cuando la red no es totalmente confiable.

3. Pruebas y Configuración del Entorno

Para la validación del sistema de publicación-suscripción se configuró un entorno virtualizado utilizando VMware Workstation con Ubuntu Linux. Las pruebas de red se ejecutaron sobre la interfaz local (loopback - 127.0.0.1), aislando el tráfico en los puertos 5000 para TCP y 5001 para UDP.

Con el objetivo de demostrar un dominio sobre la capa de transporte, el manejo de memoria y los syscalls del sistema operativo, las pruebas se dividieron en tres fases:

1. Simulación de Concurrencia en C: Utilizando hilos y la librería <sys/socket.h> para automatizar el envío de ráfagas de mensajes deportivos.

2. Ejecución Nativa en Ensamblador (x86_64) (Bono): Ejecución de binarios compilados sin librerías de alto nivel (usando el flag -no-pie), interactuando directamente con el kernel de Linux.

3. Simulación de Pérdida de Paquetes en UDP: Modificación controlada del broker en ensamblador (broker_udp_prueba.s) para descartar datagramas arbitrariamente.

La captura y análisis de la paquetería se realizó en tiempo real utilizando la herramienta Wireshark.

4. Resultados de Ejecución

4.1 Evidencia de la ejecución en TCP

En esta fase, se comprobó la confiabilidad y el control de flujo del protocolo TCP. Al ser orientado a conexión, se garantizó que los mensajes del publicador llegaran en el orden estrictamente secuencial a todos los suscriptores.

A. Ejecución en C (Concurrencia con Threads):

Se evidencia el envío automático de 12 mensajes del partido. El broker gestiona múltiples clientes simultáneamente.

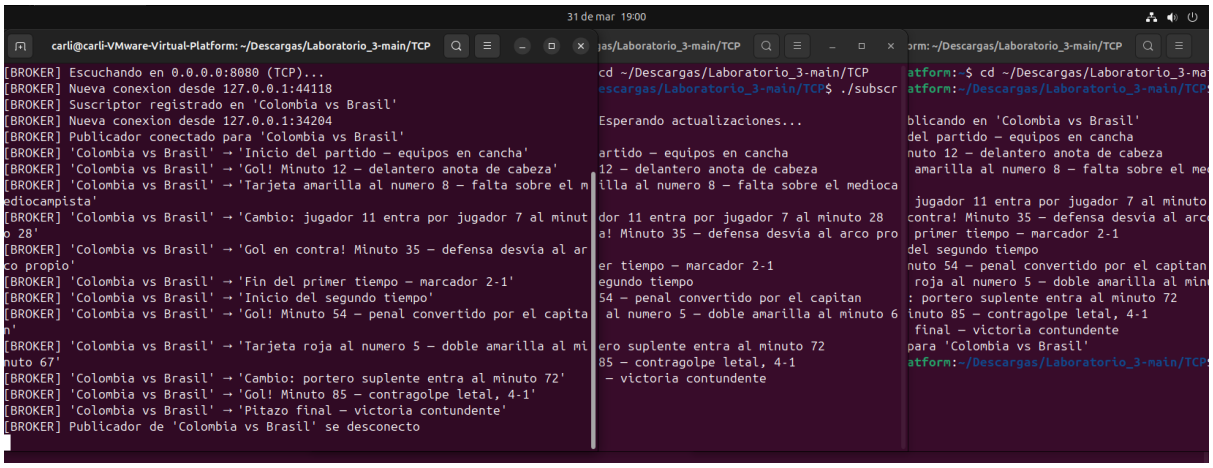

Ilustración 1: Ejecución de TCP en C: broker, publisher y subscriber en funcionamiento

B. Ejecución en Ensamblador (x86_64):

Se evidencia la misma lógica, pero implementada a bajo nivel, mostrando el uso directo de sockets mediante llamadas a funciones del sistema

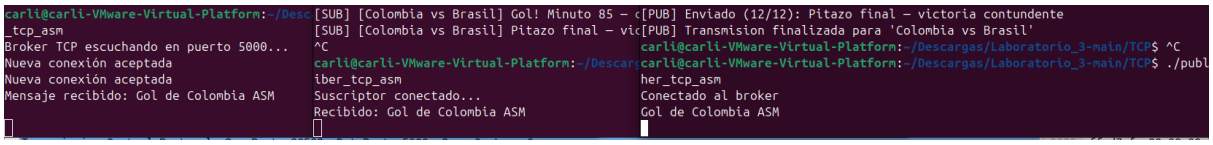

Ilustración 2: jecución de TCP en ensamblador: roker, publisher y subscriber en funcionamiento

4.2 Evidencia de la ejecución en UDP

En la prueba UDP, se evidenció la naturaleza stateless (sin estado) del protocolo. El broker distribuyó los datagramas a las direcciones registradas sin requerir un canal persistente, reduciendo la latencia, pero no garantiza la entrega.

A. Ejecución en C:

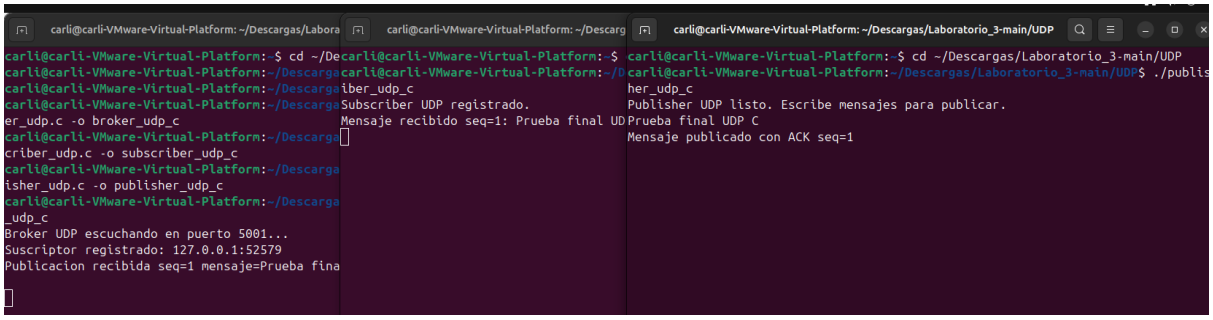

Ilustración 3: Ejecución de UDP en C: envío, recepción y confirmación de mensajes

B. Ejecución en Ensamblador (x86_64):

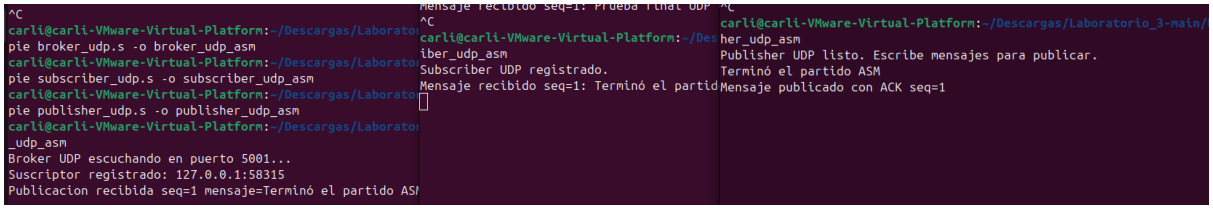

Ilustración 4: Ejecución de UDP en ensamblador (x86_64): publicación, suscripción y recepción de datagramas

4.3 Análisis de Tráfico en Wireshark

Al analizar el tráfico a bajo nivel, la diferencia arquitectónica entre ambos protocolos fue evidente:

Análisis del tráfico TCP: Se capturó exitosamente el establecimiento de la sesión. El cliente envía un paquete [SYN], el servidor responde con [SYN, ACK], y el cliente confirma con un [ACK]. Esto demuestra cómo TCP reserva recursos en el sistema operativo antes de permitir cualquier transferencia de la capa de aplicación.

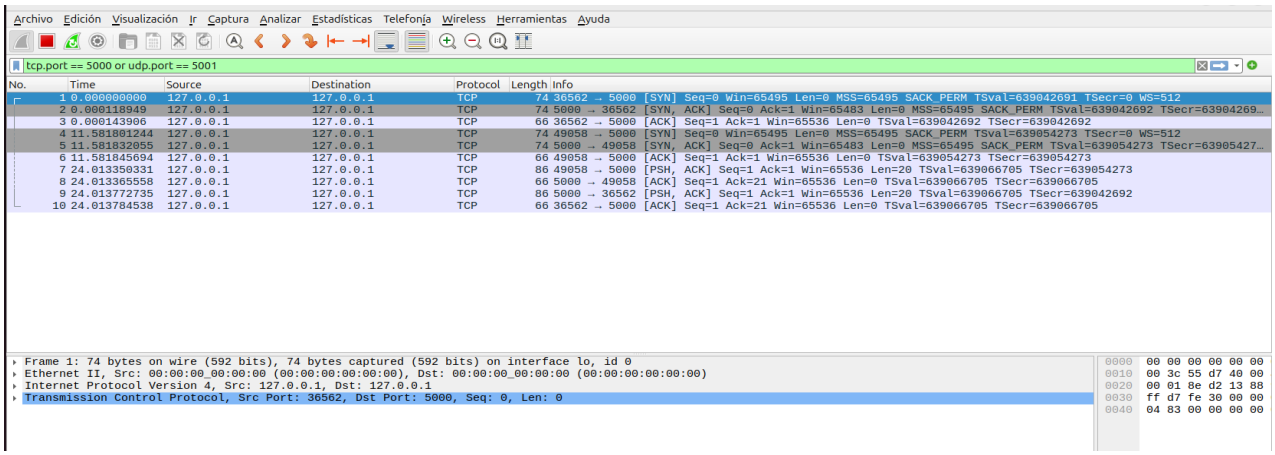

Ilustración 5: Captura de tráfico TCP en Wireshark: handshake y transmisión de datos

Análisis del tráfico UDP: Wireshark confirma la ausencia de handshake. Los datagramas con el payload (suscripción o datos) se envían directamente al puerto de destino. No se generan paquetes de control de flujo ni acuses de recibo a nivel de transporte.

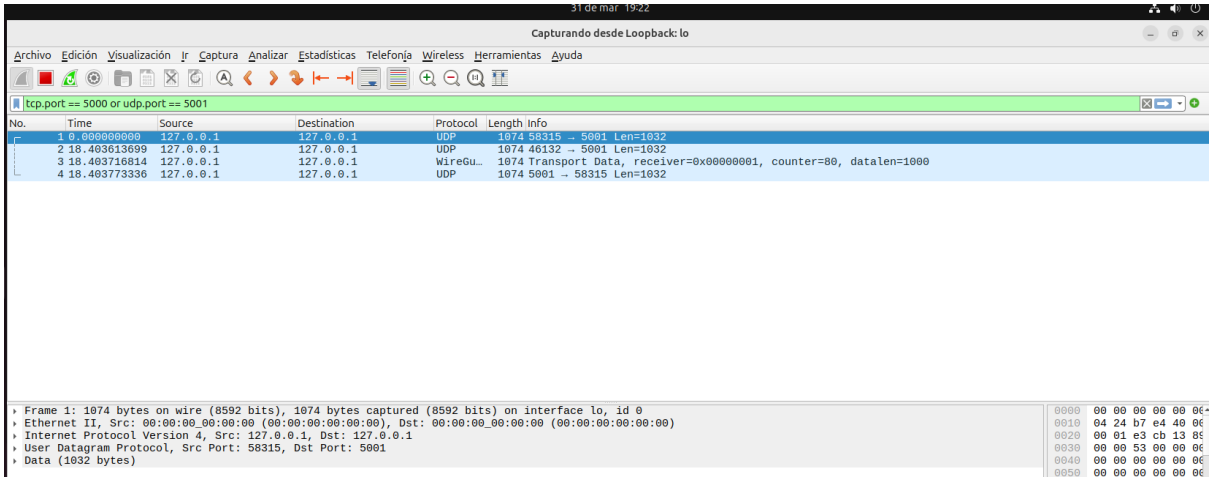

Ilustración 6: Captura de tráfico UDP en Wireshark: envío directo de datagramas sin handshake

4.4 Simulación de Pérdida de Paquetes (UDP)

Para evidenciar experimentalmente la falta de confiabilidad de UDP, se utilizó el broker modificado (broker_udp_prueba.s). Como se observa en la captura, al perderse paquetes en la red (o ser descartados por el broker), el suscriptor experimenta saltos en la secuencia de los mensajes recibidos. A diferencia de TCP, UDP no detecta perdidas ni garantiza la entrega ni retransmisión, por lo que cualquier mecanismo de confiabilidad debe implementarse a nivel de aplicación.

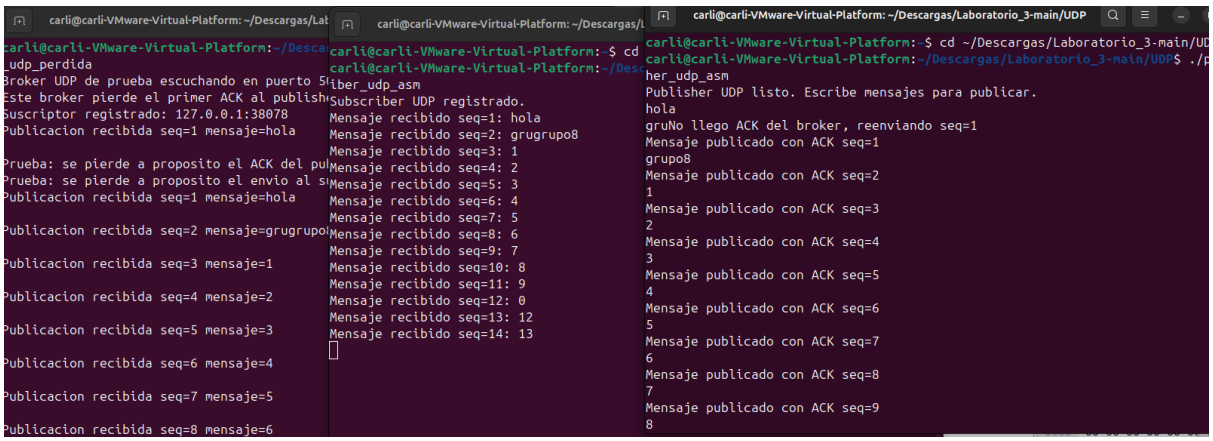

Ilustración 7: Simulación de pérdida de paquetes en UDP: reintentos del publisher y saltos en la secuencia recibida

4.5 Comparación entre TCP y UDP

Característica	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Arquitectura de Conexión	Orientado a conexión (Three-way Handshake).	Sin conexión (Connectionless / Datagramas independientes).
Confiabilidad y Garantía	Alta. Retransmite paquetes perdidos mediante ACKs.	Baja. Best-effort delivery; no garantiza entrega.
Secuenciamiento	Garantizado. Reensambla paquetes en el orden correcto.	No garantizado. Pueden llegar en desorden o duplicados.
Latencia / Sobrecarga	Alta latencia debido a cabeceras grandes y control de congestión.	Latencia mínima, ideal para tiempo real. Cabeceras ligeras.
Manejo de Estado (Broker)	Mantiene un File Descriptor persistente por cada cliente conectado.	Stateless. Usa un solo socket para recibir y reenviar iterativamente.

5. Análisis y Respuestas

• ¿Qué ocurriría si en lugar de dos publicadores (partidos transmitidos) hubiera cien partidos simultáneos? ¿Cómo impactaría esto en el desempeño del broker bajo TCP y bajo UDP?

Si hubiera 100 partidos simultaneos para el broker del TCP implicaria una mayor carga al recibir demasiadas conexiones simultaneas lo que implica un mayor uso de recursos computacionales afectando a la congestion y latencia, sin embargo, al ser TCP los mensajes siguen siendo confiables y ordenados. Por otra parte, en UDP al no manejar conexiones hay menor carga de gestion ya que el broker solo recibe y reenvia la data, lo que implica posibles pérdidas y desorden en el envió y llegada de los mensajes

• Si un gol se envía como mensaje desde el publicador y un suscriptor no lo recibe en UDP, ¿qué implicaciones tendría para la aplicación real? ¿Por qué TCP maneja mejor este escenario?

Una perdida de un gol en UDP lo que significa que el subscriptor no recibio el gol implica que el usuario vera un marcador incorrecto rompiendo la consistencia de la informacion generando confusion al espectador, TCP lo maneja mejor ya que retransmite los paquetes perdidos, garantizando la entrega y asegurando el orden de los paquetes manteniendo la consistencia de la informacion

• Compare el overhead observado en las capturas Wireshark entre TCP y UDP. ¿Cuál protocolo introduce más cabeceras por mensaje? ¿Cómo influye esto en la eficiencia

TCP introduce una sobrecarga mucho mayor. Su cabecera mínima es de 20 bytes y exige enviar múltiples paquetes de control adicionales (Handshake y ACKs). En contraste, UDP usa una cabecera de solo 8 bytes y envía un único datagrama.

Eficiencia: Para mensajes cortos (ej. "Gol"), TCP es ineficiente porque gasta más ancho de banda en control de red que en los datos útiles. UDP maximiza el rendimiento (throughput) al eliminar este tráfico extra.

• En un escenario de seguimiento en vivo de partidos, ¿qué protocolo (TCP o UDP) resultaría más adecuado? Justifique con base en los resultados de la práctica

Como podemos ver en la práctica usando UDP algunas veces se perdian los goles o llegaban desordenados, en cambio en TCP los goles llegaban pero desincronizados con el tiempo lo que generaba confusion con respecto al marcador de tiempo, para eventos críticos ejem goles se maneja TCP y para actualizaciones rapidas se utilizaria UDP.

• Si el marcador de un partido llega desordenado en UDP (por ejemplo, primero se recibe el 2–1 y luego el 1–1), ¿qué efectos tendría en la experiencia del usuario? ¿Cómo podría solucionarse este problema a nivel de aplicación?

Si los mensajes llegan desordenados generaria confusion al usuario, ya que estaria viendo un marcador incosistente lo que resulta en una mala experiencia, para solucionar este a nivel de aplicacion se pueden agregar timestamps y numeros de secuencia para ordenar los mensajes antes de mostrarlos.

• ¿Cómo cambia el desempeño del sistema cuando aumenta el número de suscriptores interesados en un mismo partido? ¿Qué diferencias se observaron entre TCP y UDP en este aspecto?

TCP escala mal: El broker degrada su rendimiento porque debe mantener un buffer de memoria por cada cliente, ejecutar la función send() de forma secuencial y esperar el ACK de cada uno. Esto retrasa la entrega para los últimos usuarios de la lista.

UDP es altamente escalable: El broker opera con un solo socket para todos. Ejecuta sendto() iterando rápidamente la lista de IPs sin esperar confirmaciones, logrando una entrega masiva y casi simultánea sin consumir memoria extra.

• ¿Qué sucede si el broker se detiene inesperadamente? ¿Qué diferencias hay entre TCP y UDP en la capacidad de recuperación de la sesión?

En primer lugar en TCP, las conexiones se cierran, los clientes detectan error por lo que implicaria volver a conectarse. En UDP, no hay conexion por lo que los clientes no saben inmediatamente el erro por lo que simplemente dejan de recibir datos, esto quiere decir que TCP facilita la deteccion de errores mientras UDP no.

• ¿Cómo garantizar que todos los suscriptores reciban en el mismo instante las actualizaciones críticas (por ejemplo, un gol)? ¿Qué protocolo facilita mejor esta sincronización y por qué?

En TCP cada conexion es independiente, puede haber diferencias de tiempo, en UDP el envio es casi simultaneo lo que es mejor para broadcasting, por lo que, UDP facilita mejor la sincronizacion

• Analice el uso de CPU y memoria en el broker cuando maneja múltiples conexiones TCP frente al manejo de datagramas UDP. ¿Qué diferencias encontró?

TCP tienee alto consumo: El sistema operativo debe reservar buffers de memoria y mantener un File Descriptor abierto por cada cliente conectado.

UDP tiene consumo mínimo: El broker centraliza todo usando un único socket para todos los clientes, requiriendo solo un arreglo ligero en memoria con las direcciones IP y puertos para saber a dónde reenviar.

6. Link GitHub

https://github.com/JuanDelgadom/Laboratorio_3.git